MOBILE APPLICATION DEVELOPMENT

COMPLETE THEORY • JAVA + XML • ACTIVITY LIFECYCLE • GENERAL SYNTAX • VIVA • ERRORS • FIX METHODS

Exam Textbook for MCA Mobile Application Development Lab

Source basis: your uploaded Android Java/XML notes, Fragment/Dynamic UI/Menus notes, lab-record material and model-question material. The source notes use Java + XML, AndroidX/AppCompatActivity and findViewById(). SQLite/database operations are excluded from this study guide.

How to use this book: Read the theory → memorize the general syntax → study Activity Lifecycle carefully → practice the reusable patterns → revise viva → use the error/fix chapter if you get stuck in the lab.

MASTER IDEA

You do not need to memorize every model question as a different program. Most questions reuse the same building blocks: XML UI → findViewById() → read values → validate → listener → store/display data → Intent/dialog/menu/list/fragment depending on the requirement.

CONTENTS

1. Android + Java/XML fundamentals
2. Project structure and Activity basics
3. Activity Lifecycle — IMPORTANT / COMPLETE
4. XML views and general widget syntax
5. Text fields, input types and validation
6. Layouts and general layout rules
7. Button clicks, Toast and AlertDialog
8. RadioButton, RadioGroup, CheckBox, Switch, RatingBar, SeekBar, ProgressBar
9. ListView, ArrayAdapter and custom adapters
10. Spinner and AutoCompleteTextView
11. ScrollView and Dynamic UI
12. Menus — Options, Context and Popup
13. Custom Dialog and PopupWindow
14. Fragments — concept, lifecycle, FragmentManager, transactions and data
15. IME / keyboard, DatePickerDialog and TimePickerDialog
16. SharedPreferences
17. Explicit and Implicit Intents — all important actions
18. General exam master architecture
19. Viva questions with detailed answers
20. Common errors and exact fixes
21. Brute-force exam recovery method
22. Final one-page revision

1. ANDROID + JAVA/XML FUNDAMENTALS

Android application development in this lab is based on a Java Activity class connected to an XML layout. XML describes the user interface; Java contains the program logic and event handling.

An Activity represents a screen in the application. Examples include Login, Home, Settings and Registration screens. The Android system manages an Activity through its lifecycle callbacks.

A View is a UI element such as TextView, EditText or Button. A ViewGroup is a container that can hold other Views, such as LinearLayout, FrameLayout or GridLayout.

A resource is an external project item such as an XML layout, string, color, drawable or menu. Android generates resource IDs such as R.id.button1 and R.layout.activity_main.

The uploaded notes use AppCompatActivity + AndroidX + findViewById() to keep the examples simple and lab-oriented.

Core Java/XML relationship:

```
activity_main.xml
      ↓
defines Button/EditText/TextView/etc.
      ↓
MainActivity.java
      ↓
setContentView(R.layout.activity_main)
      ↓
findViewById(R.id.someView)
      ↓
set listener / read value / update UI
```

General Activity syntax:

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Connect this Activity with the XML layout
        setContentView(R.layout.activity_main);
    }
}
```

Why super.onCreate()?

It calls the parent Activity implementation and allows Android/AppCompat to perform its normal Activity initialization. In an exam, keep it in the lifecycle method.

2. PROJECT STRUCTURE

Path	Purpose
app/src/main/java/...	Java source code and Activity/Fragment classes.
res/layout/	XML layouts for screens and UI.
res/values/strings.xml	Text strings and localization resources.
res/values/colors.xml	Color resources.
res/drawable/	Images, shapes and drawable resources.
res/menu/	Options/Popup menu XML files.
AndroidManifest.xml	Declares Activities, permissions, app metadata and launcher Activity.
app/build.gradle	Dependencies and Android build configuration.

General XML skeleton:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <!-- Views go here -->
```

```
</LinearLayout>
```

Important XML attributes:

Attribute	Meaning
layout_width / layout_height	Size of the View. Common values: match_parent, wrap_content.
padding	Space inside the View.
layout_margin	Space outside the View.
gravity	Alignment of content inside the View.
layout_gravity	Position of the View inside its parent.
textSize	Text size; use sp for text.
visibility	visible, invisible or gone.
hint	Placeholder shown in EditText.
layout_weight	Shares remaining LinearLayout space.

3. ACTIVITY LIFECYCLE — MOST IMPORTANT TOPIC

The Activity lifecycle is the sequence of states and callback methods through which an Activity is created, becomes visible, becomes interactive, loses focus, becomes hidden, returns and is eventually destroyed. The Android system calls these methods at specific points.

The seven callbacks required in your lab are: onCreate(), onStart(), onResume(), onPause(), onStop(), onRestart(), onDestroy().

THE MAIN SEQUENCE

```
New Activity:
onCreate()
   ↓
onStart()
   ↓
onResume()
   ↓
Activity running / user interacting

Another Activity partially covers it:
onPause()

Activity becomes completely hidden:
onStop()

Stopped Activity comes back:
onRestart()
   ↓
onStart()
   ↓
onResume()

Activity is finally destroyed:
onDestroy()
```

3.1 onCreate()

When? Called when the Activity is first created. It is the normal place to initialize the screen.

What do we normally do? setContentView(), findViewById(), initialize variables, create adapters, set listeners and read initial state.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    EditText etName = findViewById(R.id.etName);
    Button btnSave = findViewById(R.id.btnSave);

    btnSave.setOnClickListener(v -> {
        // button logic
    });
}
```

Exam sentence: onCreate() initializes the Activity, connects the XML layout and prepares the UI.

3.2 onStart()

Called when the Activity becomes visible to the user. The Activity is about to come to the foreground.

```
@Override
protected void onStart() {
    super.onStart();
    Log.d("LIFECYCLE", "onStart");
}
```

3.3 onResume()

Called when the Activity starts interacting with the user. The Activity is in the foreground and ready for user input.

```
@Override
protected void onResume() {
    super.onResume();
    Log.d("LIFECYCLE", "onResume");
}
```

3.4 onPause()

Called when another Activity or window is about to come in front. The source notes describe it as a place to save light/unsaved changes and release resources that consume CPU.

```
@Override
```

```
protected void onPause() {
    super.onPause();
    Log.d("LIFECYCLE", "onPause");
}
```

3.5 onStop()

Called when the Activity is no longer visible. The Activity is in the background.

```
@Override
protected void onStop() {
    super.onStop();
    Log.d("LIFECYCLE", "onStop");
}
```

3.6 onRestart()

Called when a stopped Activity is coming back. It is followed by onStart() and onResume().

```
@Override
protected void onRestart() {
    super.onRestart();
    Log.d("LIFECYCLE", "onRestart");
}
```

3.7 onDestroy()

Called before the Activity is destroyed. The source notes describe it as a place for cleanup of resources and held references. For important persistent data, do not depend on onDestroy() as your only saving mechanism.

```
@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d("LIFECYCLE", "onDestroy");
}
```

3.8 Complete lifecycle program for the lab

```
package com.example.myapp;

import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private static final String TAG = "LIFECYCLE";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate");
    }

    @Override
    protected void onStart() {
        super.onStart();
        Log.d(TAG, "onStart");
    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d(TAG, "onResume");
    }

    @Override
    protected void onPause() {
        super.onPause();
        Log.d(TAG, "onPause");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d(TAG, "onStop");
    }

    @Override
    protected void onRestart() {
        super.onRestart();
        Log.d(TAG, "onRestart");
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        Log.d(TAG, "onDestroy");
    }
}
```

3.9 Lifecycle sequences to memorize

Situation	Typical callback flow
Open Activity for first time	onCreate → onStart → onResume
Activity is partially covered	onPause
Activity becomes fully hidden	onStop
Stopped Activity returns	onRestart → onStart → onResume
Activity finishes	onPause → onStop → onDestroy (typical)

3.10 How to demonstrate it in the lab

Run the app and open Logcat. Filter by the TAG LIFECYCLE. Open the Activity and observe onCreate/onStart/onResume. Navigate away and observe onPause/onStop. Return to the stopped Activity and observe onRestart/onStart/onResume.

3.11 Important distinction

onPause() means the Activity is losing foreground focus. onStop() means it is no longer visible. onRestart() occurs when a previously stopped Activity is returning. onDestroy() means the Activity is being destroyed, but you should not treat it as a guaranteed location for critical persistent storage.

4. XML VIEWS AND GENERAL WIDGET SYNTAX

A View is identified in XML using android:id and retrieved in Java using findViewById().

TextView

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello"
    android:textSize="22sp" />

TextView tvTitle = findViewById(R.id.tvTitle);
tvTitle.setText("New title");
String value = tvTitle.getText().toString();
```

EditText

```
<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter name"
    android:inputType="textPersonName" />

String name = etName.getText().toString().trim();
etName.setText("Akhil");
```

Button and click

```
Button btn = findViewById(R.id.btnSubmit);

btn.setOnClickListener(v -> {
    Toast.makeText(this, "Clicked", Toast.LENGTH_SHORT).show();
});
```

Common inputType values

inputType	Use
text	Normal text.
textPersonName	Names.
textEmailAddress	Email input.
textPassword	Password input.
number / numberDecimal / numberSigned	Numeric input.
phone	Phone number.
textMultiLine	Multiline text.
date / time	Date/time entry.

5. TEXT FIELDS AND VALIDATION

Validation means checking user input before accepting or processing it.

The most common exam pattern is: read → check empty → setError/requestFocus → return → continue if valid.

```
String name = etName.getText().toString().trim();

if (name.isEmpty()) {
    etName.setError("Name is required");
    etName.requestFocus();
    return;
}
```

Email validation:

```
private boolean isValidEmail(String email) {
    return android.util.Patterns.EMAIL_ADDRESS
            .matcher(email)
            .matches();
}
```

Indian 10-digit mobile pattern from the notes:

```
private boolean isValidPhone(String phone) {
    return phone.matches("[6-9][0-9]{9}");
```

```
}
```

Strong password pattern from the notes:

```
private boolean isStrongPassword(String pwd) {
    return pwd.matches(
        "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=!]).{8,}$"
    );
}
```

Typical validation chain:

```
if (name.isEmpty()) {
    etName.setError("Name required");
    etName.requestFocus();
} else if (!isValidEmail(email)) {
    etEmail.setError("Enter valid email");
    etEmail.requestFocus();
} else if (!isValidPhone(phone)) {
    etPhone.setError("Enter valid mobile");
    etPhone.requestFocus();
} else {
    Toast.makeText(this,
        "Valid input",
        Toast.LENGTH_SHORT).show();
}
```

6. LAYOUTS — THEORY + GENERAL SYNTAX

LinearLayout: arranges children vertically or horizontally.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <!-- children -->

</LinearLayout>
```

RelativeLayout: positions views relative to the parent or other views. It appears in the original notes as a common layout pattern.

ConstraintLayout: positions views using constraints. Useful for responsive layouts and appears in the practice/model questions.

FrameLayout: stacks children; commonly used as a container for fragments.

```
<FrameLayout
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

TableLayout: organizes content into rows using TableRow. Useful for structured forms.

```
<TableLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="1">

    <TableRow>
        <TextView android:text="Name" />
        <EditText android:hint="Enter name" />
    </TableRow>

</TableLayout>
```

GridLayout: arranges children in rows and columns.

```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="2">

    <!-- buttons/views -->

</GridLayout>
```

ScrollView: allows vertical scrolling. It normally contains one direct child, commonly a LinearLayout.

```
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- many views -->

    </LinearLayout>
</ScrollView>
```

Common layout mistake: putting multiple direct children inside a ScrollView. Use one parent container and place the other Views inside it.

7. TOAST, ALERTDIALOG AND BUTTON ACTIONS

Toast: small temporary message that disappears automatically and does not take focus.

```
Toast.makeText(
    this,
    "Saved!",
    Toast.LENGTH_SHORT
).show();
```

AlertDialog: modal dialog used for confirmation, messages or choices.

```
new AlertDialog.Builder(this)
    .setTitle("Confirm")
    .setMessage("Do you want to continue?")
    .setPositiveButton("Yes", (dialog, which) -> {
        // Yes action
    })
```

```
    .setNegativeButton("No", null)
    .show();
```

Single-choice AlertDialog:

```
String[] modes = {"Light", "Dark", "Blue"};

new AlertDialog.Builder(this)
    .setTitle("Choose Theme")
    .setSingleChoiceItems(modes, 0,
        (dialog, which) -> {
            Toast.makeText(this,
                modes[which],
                Toast.LENGTH_SHORT).show();
            dialog.dismiss();
        })
    .show();
```

RadioButton vs CheckBox

RadioButton	CheckBox
Usually one choice within a RadioGroup.	Multiple independent choices can be selected.
Read using getCheckedRadioButtonId().	Read using isChecked().

8. IMPORTANT FORM WIDGETS

RadioGroup + RadioButton:

```
int id = rgGender.getCheckedRadioButtonId();

if (id == -1) {
    Toast.makeText(this,
        "Select gender",
        Toast.LENGTH_SHORT).show();
} else {
    RadioButton rb = findViewById(id);
    String gender = rb.getText().toString();
}
```

CheckBox:

```
if (cbJava.isChecked()) {
    // Java selected
}

if (cbPython.isChecked()) {
    // Python selected
}
```

Switch:

```
swNotifications.setOnCheckedChangeListener(
    (buttonView, isChecked) -> {
        Toast.makeText(this,
            isChecked ? "Enabled" : "Disabled",
            Toast.LENGTH_SHORT).show();
    }
);
```

RatingBar:

```
float rating = ratingBar.getRating();

ratingBar.setOnRatingBarChangeListener(
    (bar, value, fromUser) -> {
        Toast.makeText(this,
            "Rating: " + value,
            Toast.LENGTH_SHORT).show();
    }
);
```

SeekBar:

```
seekBar.setOnSeekBarChangeListener(
    new SeekBar.OnSeekBarChangeListener() {

        @Override
        public void onProgressChanged(
                SeekBar bar,
                int progress,
                boolean fromUser) {
            // use progress
        }

        @Override public void onStartTrackingTouch(SeekBar bar) {}
```

```
        @Override public void onStopTrackingTouch(SeekBar bar) {}
    }
);
```

ProgressBar:

```
progressBar.setProgress(60);
progressBar.setVisibility(View.GONE);
```

9. LISTVIEW + ARRAYADAPTER

A ListView displays a list of items. An Adapter acts as the bridge between the data and the ListView.

The simplest exam pattern uses an ArrayList and ArrayAdapter.

```
ArrayList<String> items = new ArrayList<>();

items.add("Java");
items.add("Android");
items.add("XML");

ArrayAdapter<String> adapter =
        new ArrayAdapter<>(
            this,
            android.R.layout.simple_list_item_1,
            items
        );

ListView listView = findViewById(R.id.listView);

listView.setAdapter(adapter);
```

Item click:

```
listView.setOnItemClickListener(
    (parent, view, position, id) -> {

        String selected =
                items.get(position);

        Toast.makeText(
            this,
            selected,
            Toast.LENGTH_SHORT
        ).show();
    }
);
```

Adding and deleting items:

```
items.add("New item");
adapter.notifyDataSetChanged();

items.remove(position);
adapter.notifyDataSetChanged();
```

Why notifyDataSetChanged()?

It tells the Adapter that the underlying data has changed and the ListView should refresh its displayed contents.

Custom Adapter concept:

For custom rows, create a row XML and a custom Adapter extending BaseAdapter or another suitable adapter class. In getView(), inflate/reuse the row, find its views, put the data into them and return the row View.

```
@Override
public View getView(
        int position,
        View convertView,
        ViewGroup parent) {

    if (convertView == null) {
        convertView = LayoutInflater.from(context)
            .inflate(R.layout.row_student, parent, false);
    }

    TextView tvName =
        convertView.findViewById(R.id.tvName);

    Student s = list.get(position);

    tvName.setText(s.getName());

    return convertView;
}
```

10. SPINNER + AUTOCOMPLETETEXTVIEW

Spinner is a drop-down selection control.

```
String[] courses = {
    "MCA", "MBA", "BCA", "MSc"
};

Spinner spinner = findViewById(R.id.spinner);
```

```
ArrayAdapter<String> adapter =
    new ArrayAdapter<>(
        this,
        android.R.layout.simple_spinner_item,
        courses
    );

adapter.setDropDownViewResource(
    android.R.layout.simple_spinner_dropdown_item
);

spinner.setAdapter(adapter);

spinner.setOnItemSelectedListener(
    new AdapterView.OnItemSelectedListener() {

        @Override
        public void onItemSelected(
                AdapterView<?> parent,
                View view,
                int position,
                long id) {

            String value = courses[position];
        }

        @Override
        public void onNothingSelected(
                AdapterView<?> parent) {
        }
    }
);
```

AutoCompleteTextView:

```
String[] cities = {
    "Kochi", "Kollam", "Kannur"
};

ArrayAdapter<String> adapter =
    new ArrayAdapter<>(
        this,
        android.R.layout.simple_dropdown_item_1line,
        cities
    );

AutoCompleteTextView actv =
    findViewById(R.id.cityAutoCompleteTextView);

actv.setAdapter(adapter);
actv.setThreshold(1);
```

11. SCROLLVIEW + DYNAMIC UI + TEXTWATCHER

Dynamic UI means creating/modifying/removing Views in Java at runtime instead of defining every View in XML.

```
LinearLayout container =
    findViewById(R.id.container);

TextView tv = new TextView(this);

tv.setText("Created in Java");
tv.setTextSize(20);
tv.setPadding(16, 16, 16, 16);

container.addView(tv);
```

Dynamic EditText:

```
EditText field = new EditText(this);

field.setHint("Enter value");

LinearLayout.LayoutParams params =
    new LinearLayout.LayoutParams(
        ViewGroup.LayoutParams.MATCH_PARENT,
        ViewGroup.LayoutParams.WRAP_CONTENT
    );

field.setLayoutParams(params);
container.addView(field);
```

Remove Views:

```
container.removeView(view);
container.removeAllViews();

int count = container.getChildCount();
View child = container.getChildAt(0);
```

TextWatcher for live character count:

```
editText.addTextChangedListener(
    new TextWatcher() {

        @Override
        public void beforeTextChanged(
                CharSequence s,
                int start,
                int count,
                int after) {
        }

        @Override
        public void onTextChanged(
                CharSequence s,
                int start,
                int before,
                int count) {

            textView.setText(
                "Characters: " + s.length()
            );
        }

        @Override
        public void afterTextChanged(
                Editable s) {
        }
    }
);
```

12. MENUS — OPTIONS, CONTEXT AND POPUP

Android traditional View applications use three important menu patterns in your notes: Options Menu, Context Menu and Popup Menu.

Options Menu = Activity-level actions/settings. Context Menu = actions for a selected View, usually after long press. Popup Menu = small contextual menu anchored to a View.

Options Menu XML:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/action_settings"
        android:title="Settings" />

    <item
        android:id="@+id/action_save"
        android:title="Save" />

</menu>
```

Options Menu Java:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(
        R.menu.main_menu,
        menu
    );
    return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {

    if (item.getItemId() == R.id.action_save) {
        Toast.makeText(this,
            "Saved",
            Toast.LENGTH_SHORT).show();
        return true;
    }

    return super.onOptionsItemSelected(item);
}
```

Context Menu:

```
registerForContextMenu(listView);

@Override
public void onCreateContextMenu(
        ContextMenu menu,
        View v,
        ContextMenu.ContextMenuInfo info) {

    super.onCreateContextMenu(menu, v, info);

    menu.setHeaderTitle("Choose Action");
    menu.add(0, 1, 0, "Edit");
    menu.add(0, 2, 1, "Delete");
}

@Override
public boolean onContextItemSelected(MenuItem item) {

    if (item.getItemId() == 2) {
        // delete
        return true;
    }

    return super.onContextItemSelected(item);
}
```

Popup Menu:

```
Button more = findViewById(R.id.btnMore);

more.setOnClickListener(v -> {

    PopupMenu popup =
        new PopupMenu(MainActivity.this, v);

    popup.getMenuInflater().inflate(
        R.menu.popup_menu,
        popup.getMenu()
    );

    popup.setOnMenuItemClickListener(item -> {
```

```
        Toast.makeText(
            this,
            item.getTitle(),
            Toast.LENGTH_SHORT
        ).show();
        return true;
    });

    popup.show();
});
```

Menu comparison:

Menu	Purpose	Trigger
Options Menu	Activity-level actions/settings.	App bar / overflow.
Context Menu	Actions for a selected View.	Usually long press.
Popup Menu	Small contextual actions.	Tap an anchor View.

Dynamic menu state:

```
@Override
public boolean onPrepareOptionsMenu(Menu menu) {
    MenuItem save = menu.findItem(R.id.action_save);
    save.setVisible(isFormValid);
    return super.onPrepareOptionsMenu(menu);
}

// Call when menu state changes:
invalidateOptionsMenu();
```

13. CUSTOM DIALOG + POPUPWINDOW

A custom dialog uses a custom XML layout inside an AlertDialog. It is useful for forms such as Add Item.

```
View view = getLayoutInflater().inflate(
    R.layout.dialog_add_item,
    null
);

EditText etName =
    view.findViewById(R.id.etName);

new AlertDialog.Builder(this)
    .setTitle("Add Item")
    .setView(view)
    .setPositiveButton("Save",
        (dialog, which) -> {
            String name =
                etName.getText().toString();
        })
    .setNegativeButton("Cancel", null)
    .show();
```

PopupWindow is a custom floating View. The notes show showAsDropDown(anchor) and showAtLocation(...).

```
View popupView =
    getLayoutInflater().inflate(
        R.layout.popup_layout, null);

PopupWindow popupWindow =
    new PopupWindow(
        popupView,
        ViewGroup.LayoutParams.WRAP_CONTENT,
        ViewGroup.LayoutParams.WRAP_CONTENT,
        true
    );

popupWindow.showAsDropDown(
    button, 0, 10
);
```

14. FRAGMENTS — COMPLETE THEORY

A Fragment is a reusable portion of UI with its own layout and lifecycle, hosted inside an Activity. Fragments are useful for flexible screens, navigation and reusable sections.

Your uploaded notes use AndroidX Fragment classes for their main Fragment examples.

Fragment lifecycle from the notes:

```
onAttach()
   ↓
onCreate()
   ↓
onCreateView()
   ↓
onViewCreated()
   ↓
onStart()
   ↓
onResume()
   ↓
onPause()
   ↓
onStop()
   ↓
onDestroyView()
   ↓
onDestroy()
   ↓
onDetach()
```

Meaning:

Callback	Purpose
onAttach()	Fragment is attached to the Activity/context.
onCreate()	Initialize non-UI state.
onCreateView()	Inflate/create and return the Fragment layout.
onViewCreated()	View hierarchy is ready; find views and attach listeners.
onStart()/onResume()	Visible / active and interactive.
onPause()/onStop()	Leaving foreground / no longer visible.
onDestroyView()	Fragment's View hierarchy is destroyed.
onDestroy()	Fragment object is being destroyed.
onDetach()	Fragment is detached from Activity.

Basic Fragment:

```
public class HomeFragment extends Fragment {

    public HomeFragment() {
        // Empty constructor
    }

    @Override
    public View onCreateView(
            LayoutInflater inflater,
            ViewGroup container,
            Bundle savedInstanceState) {

        return inflater.inflate(
            R.layout.fragment_home,
            container,
            false
        );
    }
}
```

Dynamic Fragment container:

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/container"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Replace Fragment:

```
getSupportFragmentManager()
```

```
    .beginTransaction()
    .replace(
        R.id.container,
        new HomeFragment()
    )
    .commit();
```

Back stack:

```
getSupportFragmentManager()
    .beginTransaction()
    .replace(
        R.id.container,
        new ProfileFragment()
    )
    .addToBackStack(null)
    .commit();
```

Why addToBackStack()?

It records the Fragment transaction so the user can use Back to return to the previous Fragment state.

Fragment data using Bundle:

```
Bundle bundle = new Bundle();
bundle.putString("name", "Akhil");

DetailsFragment fragment =
    new DetailsFragment();

fragment.setArguments(bundle);

getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.container, fragment)
    .commit();
```

Receive it:

```
Bundle bundle = getArguments();

if (bundle != null) {
    String name =
        bundle.getString("name");
}
```

Important exam distinction: Activity-to-Activity data is commonly passed through Intent extras/Bundle. Activity-to-Fragment data is commonly passed through Fragment arguments using setArguments()/getArguments().

15. IME / KEYBOARD + DATE + TIME

IME means Input Method Editor, i.e. the on-screen keyboard.

IME options change the action button shown on the keyboard, such as Search, Done, Next, Go, Send or Previous.

```
<EditText
    android:id="@+id/etSearch"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Search..."
    android:inputType="text"
    android:imeOptions="actionSearch"
    android:maxLines="1" />

etSearch.setOnEditorActionListener(
    (v, actionId, event) -> {

        if (actionId ==
                EditorInfo.IME_ACTION_SEARCH) {

            String q =
                etSearch.getText().toString();

            Toast.makeText(
                this,
                "Searching: " + q,
                Toast.LENGTH_SHORT
            ).show();

            return true;
        }

        return false;
    }
);
```

Hide keyboard:

```
InputMethodManager imm =
    (InputMethodManager)
        getSystemService(Context.INPUT_METHOD_SERVICE);

imm.hideSoftInputFromWindow(
    etSearch.getWindowToken(),
    0
);
```

DatePickerDialog:

```
Calendar calendar = Calendar.getInstance();

DatePickerDialog dialog =
    new DatePickerDialog(
        this,
        (view, year, month, day) -> {

            String date =
                day + "/" + (month + 1) + "/" + year;

            tvDate.setText(date);
        },
        calendar.get(Calendar.YEAR),
        calendar.get(Calendar.MONTH),
        calendar.get(Calendar.DAY_OF_MONTH)
    );

dialog.show();
```

Prevent past dates when required:

```
dialog.getDatePicker().setMinDate(
    System.currentTimeMillis()
);
```

TimePickerDialog:

```
Calendar calendar = Calendar.getInstance();

TimePickerDialog dialog =
    new TimePickerDialog(
        this,
        (view, hour, minute) -> {

            String time =
                String.format(
                    Locale.getDefault(),
                    "%02d:%02d",
                    hour,
                    minute
                );

            tvTime.setText(time);
        },
        calendar.get(Calendar.HOUR_OF_DAY),
        calendar.get(Calendar.MINUTE),
        true
    );

dialog.show();
```

16. SHAREDPREFERENCES

SharedPreferences stores small amounts of persistent application data as key-value pairs. Typical lab uses include saving a name, email, remember-me flag or login session state.

Create/open preferences:

```
SharedPreferences sp =
    getSharedPreferences(
        "UserData",
        MODE_PRIVATE
    );
```

Write data:

```
sp.edit()
    .putString("name", "Akhil")
    .putString("email", "a@example.com")
    .putBoolean("remember", true)
    .apply();
```

Read data:

```
String name =
    sp.getString("name", "Not saved");

boolean remember =
    sp.getBoolean("remember", false);
```

Clear all data:

```
sp.edit().clear().apply();
```

Remember-me login pattern:

```
SharedPreferences sp =
    getSharedPreferences(
        "Session",
        MODE_PRIVATE
    );

if (sp.getBoolean("isLoggedIn", false)) {
    startActivity(
        new Intent(this, HomeActivity.class)
    );
    finish();
    return;
}

// after successful login:
sp.edit()
    .putBoolean("isLoggedIn", true)
    .putString("user", username)
    .apply();
```

Security note from the source: do not store real passwords in plain SharedPreferences. For sensitive data, the notes recommend appropriate secure storage such as encrypted preferences/Android Keystore. For your exam, SharedPreferences is primarily a small key-value persistence concept.

17. INTENTS — EXPLICIT + IMPLICIT

An Intent is a messaging object used to request an action from another Android application component. The notes distinguish Explicit and Implicit Intents.

Explicit Intent:

The exact target component class is named. Typical use: Activity → Activity.

```
Intent intent = new Intent(
    MainActivity.this,
    WelcomeActivity.class
);

intent.putExtra("user", user);

startActivity(intent);
```

Receive:

```
String user =
    getIntent().getStringExtra("user");
```

Bundle version:

```
Bundle bundle = new Bundle();
bundle.putString("user", user);

Intent intent =
    new Intent(
        MainActivity.this,
        WelcomeActivity.class
    );

intent.putExtras(bundle);
startActivity(intent);
```

Implicit Intent:

The action/data is described, and Android finds a suitable application/component that can handle it.

Browser / Website:

```
Intent intent = new Intent(
    Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com")
);

if (intent.resolveActivity(
         getPackageManager()) != null) {

    startActivity(intent);
}
```

Dialer:

```
Intent intent = new Intent(
    Intent.ACTION_DIAL,
    Uri.parse("tel:9876543210")
);

if (intent.resolveActivity(
         getPackageManager()) != null) {
    startActivity(intent);
}
```

SMS:

```
Intent intent = new Intent(
    Intent.ACTION_SENDTO,
    Uri.parse("sms:9876543210")
);

if (intent.resolveActivity(
         getPackageManager()) != null) {
    startActivity(intent);
}
```

Email:

```
Intent intent =
    new Intent(Intent.ACTION_SENDTO);

intent.setData(
    Uri.parse("mailto:example@gmail.com")
);

intent.putExtra(
    Intent.EXTRA_SUBJECT,
    "MAD"
);

if (intent.resolveActivity(
         getPackageManager()) != null) {
    startActivity(intent);
}
```

Maps:

```
Uri location =
    Uri.parse("geo:0,0?q=Kochi");

Intent intent =
    new Intent(
        Intent.ACTION_VIEW,
        location
    );

if (intent.resolveActivity(
         getPackageManager()) != null) {
    startActivity(intent);
}
```

Calendar:

```
Intent intent =
    new Intent(Intent.ACTION_INSERT);

intent.setData(
    Uri.parse(
        "content://com.android.calendar/events"
    )
);

if (intent.resolveActivity(
        getPackageManager()) != null) {
    startActivity(intent);
}
```

Share text:

```
Intent intent =
    new Intent(Intent.ACTION_SEND);

intent.setType("text/plain");

intent.putExtra(
    Intent.EXTRA_TEXT,
    "Hello from MAD"
);

if (intent.resolveActivity(
        getPackageManager()) != null) {

    startActivity(
        Intent.createChooser(
            intent,
            "Share using"
        )
    );
}
```

Why resolveActivity()?

It checks whether an installed application can handle the Intent before calling startActivity(). This avoids an ActivityNotFoundException when no suitable application exists.

18. EXAM MASTER ARCHITECTURE

The uploaded question bank explicitly identifies a reusable architecture behind the integrated model questions.

```
ACTIVITY 1
   ↓
Form widgets
   ↓
Validation
   ↓
Options Menu / Button
   ↓
AlertDialog
   ↓
SharedPreferences
   ↓
Bundle
   ↓
Explicit Intent
   ↓
ACTIVITY 2
   ↓
Read Intent data / preferences
   ↓
ListView + Adapter
   ↓
Normal click
   → AlertDialog / Toast
   ↓
Long press
   → Context Menu
   ↓
Implicit Intent
   → Browser / Dialer / Email / Maps / Calendar
```

Reusable replacement rule:

Keep the pattern	Change for the question
EditText + validation	Name, email, patient, destination, product, username, etc.
RadioGroup	Gender, intensity, room type, category, etc.
CheckBox	Hobbies, facilities, warnings, features, add-ons.
Spinner	Course, state, country, vehicle, room, category.
ListView + adapter	Products, tasks, contacts, courses, bookings, books.
Intent destination	Browser, dialer, SMS, email, maps, calendar.
Messages	Question-specific validation, confirmation and Toast text.

> This is why learning patterns is more efficient than memorizing dozens of unrelated programs.

19. VIVA QUESTIONS + DETAILED ANSWERS

These answers are designed to be spoken in a viva: first give the direct answer, then add the explanation if the examiner asks further.

Q1. What is an Activity?

Answer: An Activity represents a screen in an Android application. It has a lifecycle managed by the Android system and can contain Views such as TextView, EditText and Button.

Q2. What is the Activity lifecycle?

Answer: It is the sequence of callback methods through which an Activity is created, becomes visible, becomes interactive, loses focus, becomes hidden, returns and is destroyed. The important callbacks are onCreate, onStart, onResume, onPause, onStop, onRestart and onDestroy.

Q3. Which method is normally used to set the XML layout?

Answer: setContentView(R.layout.activity_main) is normally called in onCreate().

Q4. Difference between onPause() and onStop()?

Answer: onPause() occurs when the Activity is losing foreground focus. onStop() occurs when the Activity is no longer visible.

Q5. When is onRestart() called?

Answer: When a previously stopped Activity is coming back. It is followed by onStart() and onResume().

Q6. What is Log.d() used for?

Answer: It writes debug messages to Logcat. It is useful for observing lifecycle callbacks and debugging program flow.

Q7. What is findViewById()?

Answer: It finds a View from the currently loaded XML layout using its resource ID and returns a reference that Java can use.

Q8. What is the difference between match_parent and wrap_content?

Answer: match_parent asks the View to use the available parent dimension; wrap_content asks it to use only the space required by its content.

Q9. Why use dp and sp?

Answer: dp is commonly used for dimensions and spacing; sp is used for text size so text can respect user font scaling.

Q10. What is validation?

Answer: Validation checks whether user input satisfies the required conditions before processing it.

Q11. What does setError() do?

Answer: It displays an error indicator/message associated with the input field. requestFocus() can move focus to that field.

Q12. RadioButton vs CheckBox?

Answer: RadioButtons inside a RadioGroup normally provide one choice; CheckBoxes allow independent multiple selections.

Q13. What does getCheckedRadioButtonId() return?

Answer: It returns the ID of the selected RadioButton. If none is selected, it returns -1.

Q14. What is an Adapter?

Answer: An Adapter connects data to a UI component such as ListView or Spinner and supplies the Views/items that should be displayed.

Q15. Why notifyDataSetChanged()?

Answer: After changing the underlying list, notifyDataSetChanged() tells the Adapter that the data changed so the displayed list can refresh.

Q16. What is a Spinner?

Answer: A Spinner is a drop-down selection widget. It is commonly populated using an ArrayAdapter.

Q17. What is ScrollView?

Answer: It provides scrolling for content larger than the available screen area. A common pattern is one LinearLayout child containing many Views.

Q18. What is dynamic UI?

Answer: Creating, modifying or removing Views programmatically at runtime rather than defining every View in XML.

Q19. What is an Intent?

Answer: An Intent is a messaging object used to request an action from another Android component.

Q20. Explicit vs Implicit Intent?

Answer: Explicit Intent names the exact target component, such as WelcomeActivity. Implicit Intent describes an action/data and lets Android find a suitable handler, such as a browser for a website.

Q21. Why use resolveActivity()?

Answer: To check whether there is an installed component capable of handling the Intent before starting it.

Q22. What is Bundle?

Answer: Bundle is a key-value container commonly used to carry small pieces of data such as String/int values between Android components.

Q23. What is SharedPreferences?

Answer: A simple persistent key-value storage mechanism for small application data.

Q24. Why use apply()?

Answer: It commits SharedPreferences changes asynchronously. It is commonly used in the lab examples after editing preferences.

Q25. What is a Fragment?

Answer: A Fragment is a reusable portion of UI with its own lifecycle, hosted inside an Activity.

Q26. replace() vs addToBackStack()?

Answer: replace() changes the Fragment displayed in a container. addToBackStack() records the transaction so Back can return to the previous Fragment state.

Q27. setArguments() / getArguments()?

Answer: setArguments() attaches a Bundle to a Fragment before it is displayed. getArguments() retrieves that Bundle inside the Fragment.

Q28. Options Menu vs Context Menu vs Popup Menu?

Answer: Options Menu is for Activity-level actions, Context Menu is usually triggered by long press on a View, and Popup Menu is a small menu anchored to a View.

Q29. What is TextWatcher?

Answer: It observes text changes in an editable text field. It provides beforeTextChanged(), onTextChanged() and afterTextChanged().

Q30. What is IME?

Answer: Input Method Editor, referring to the on-screen keyboard/input method. imeOptions can control actions such as Search or Done.

Q31. DatePickerDialog vs TimePickerDialog?

Answer: DatePickerDialog lets the user choose a date; TimePickerDialog lets the user choose a time.

Q32. Why use finish() after redirecting from Login?

Answer: It removes the current Activity from the back stack so the user does not return to the login screen using Back.

Q33. What is an AlertDialog used for?

Answer: Confirmation, messages, single-choice/multi-choice selection and simple user decisions.

Q34. What is a Toast?

Answer: A short temporary message that disappears automatically.

Q35. What is the purpose of AndroidManifest.xml?

Answer: It declares application components and metadata such as Activities and required permissions, and identifies the launcher Activity.

20. COMMON ERRORS + EXACT FIX METHODS

This section is designed for the practical exam. If the program does not work, identify the category first instead of rewriting everything.

ERROR 1 — Cannot find symbol R.id.xyz

Cause: XML ID is different, missing, or the resource XML has another error.

Fix: Check the exact spelling in XML and Java. Make sure the View has android:id="@+id/xyz". Fix any red XML error first, then rebuild.

```
XML:
android:id="@+id/btnSubmit"

Java:
findViewById(R.id.btnSubmit);
```

ERROR 2 — NullPointerException after findViewById()

Cause: Wrong ID, wrong layout loaded, or findViewById() is called before setContentView().

Fix: setContentView() first → then findViewById(). Verify the ID exists in that exact layout.

```
setContentView(R.layout.activity_main);

Button btn =
    findViewById(R.id.btnSubmit);
```

ERROR 3 — Activity cannot be opened

Cause: The second Activity may not be declared correctly in the manifest or the class name/package may be wrong.

Fix: Check AndroidManifest.xml and the Activity class name/package.

```
<activity
    android:name=".WelcomeActivity"
    android:exported="false" />
```

ERROR 4 — App crashes when opening website/dialer/email

Cause: No installed application can handle the Intent.

Fix: Use resolveActivity() before startActivity().

```
if (intent.resolveActivity(
        getPackageManager()) != null) {
    startActivity(intent);
} else {
    Toast.makeText(
        this,
        "No suitable app found",
        Toast.LENGTH_SHORT
    ).show();
}
```

ERROR 5 — RadioButton value not received

Cause: No RadioButton is selected, or the selected ID is not checked.

Fix: Test for -1 first.

```
int id =
    radioGroup.getCheckedRadioButtonId();

if (id == -1) {
    Toast.makeText(
        this,
        "Select one option",
        Toast.LENGTH_SHORT
    ).show();
    return;
}

RadioButton rb = findViewById(id);
```

ERROR 6 — ListView does not update after delete/add

Cause: The ArrayList changed but the Adapter was not notified.

Fix: Call adapter.notifyDataSetChanged().

```
items.remove(position);
```

```
adapter.notifyDataSetChanged();
```

ERROR 7 — ListView is empty

Cause: Items were not added, adapter was not created correctly, or setAdapter() was forgotten.

Fix: Verify this order: create list → add items → create adapter → setAdapter().

ERROR 8 — Spinner not showing values

Cause: Adapter not attached.

Fix: spinner.setAdapter(adapter). Also set the drop-down resource if required.

ERROR 9 — Spinner callback not working

Cause: OnItemSelectedListener not attached or wrong callback implementation.

Fix: Implement both onItemSelected() and onNothingSelected().

ERROR 10 — ScrollView content not scrolling

Cause: Wrong height/structure or multiple direct children.

Fix: Use one direct child such as a vertical LinearLayout with wrap_content height.

ERROR 11 — Dynamic View does not appear

Cause: View was created but never added to a visible parent.

Fix: configure View → optional LayoutParams → parent.addView(view).

ERROR 12 — Fragment crashes because view is null

Cause: findViewById() was performed on the wrong object or before the Fragment view was created.

Fix: Inflate the Fragment layout in onCreateView() and use the returned view / use onViewCreated().

```
@Override
public View onCreateView(
        LayoutInflater inflater,
        ViewGroup container,
        Bundle savedInstanceState) {

    View view = inflater.inflate(
        R.layout.fragment_home,
        container,
        false
    );

    TextView tv =
        view.findViewById(R.id.tvTitle);

    return view;
}
```

ERROR 13 — Fragment transaction does nothing

Cause: Wrong container ID, transaction not committed, or mixed Fragment APIs.

Fix: Use a valid FragmentContainerView/FrameLayout ID, call commit(), and do not mix android.app.Fragment with androidx.fragment.app.Fragment.

ERROR 14 — SharedPreferences value not saved

Cause: Wrong key, wrong preference file, or edit() was not applied.

Fix: Check getSharedPreferences name and keys. End writes with apply().

ERROR 15 — getString() returns default value

Cause: The key was never saved or the key name does not exactly match.

Fix: Compare the putString() key and getString() key character-by-character.

ERROR 16 — TextWatcher causes unexpected behavior

Cause: Validation/update code may run on every keystroke.

Fix: Keep live-update logic small; choose the correct callback. Use afterTextChanged() when you need the final Editable value.

ERROR 17 — DatePicker allows past dates

Fix: Apply setMinDate(System.currentTimeMillis()) when the question requires future dates only.

ERROR 18 — IME Done/Search not triggering

Cause: imeOptions and action ID do not match.

Fix: XML actionSearch/actionDone must match EditorInfo.IME_ACTION_SEARCH/IME_ACTION_DONE.

ERROR 19 — App does not compile after adding a menu

Cause: Menu XML ID does not exist or Java references a different ID.

Fix: Compare res/menu XML IDs with R.id references. Also ensure the file is inside res/menu/.

ERROR 20 — Wrong FragmentManager import

Cause: Mixing platform and AndroidX Fragment APIs.

Fix: If using AndroidX Fragment, use androidx.fragment.app.Fragment, FragmentManager and FragmentTransaction consistently.

21. BRUTE-FORCE EXAM RECOVERY METHOD

If you are in the practical exam and your program is not working, do not panic. Use this deterministic recovery sequence.

STEP 1 — Make the app compile first

Remove optional features temporarily. Keep only Activity + XML + one Button. Build/run. Once it works, add the required feature.

STEP 2 — Check the three basic lines

```
setContentView(R.layout.activity_main);

Button btn =
    findViewById(R.id.btnSubmit);

btn.setOnClickListener(v -> {
    Toast.makeText(
        this,
        "Working",
        Toast.LENGTH_SHORT
    ).show();
});
```

If this works, your Activity/XML connection is correct.

STEP 3 — Add one concept at a time

```
Form
→ validation
→ adapter
→ dialog
→ SharedPreferences
→ Intent
→ second Activity
→ menu
→ long click
→ implicit Intent
→ fragment
```

STEP 4 — If IDs fail

Copy the ID directly from XML and use R.id.. Never type a similar name from memory.

STEP 5 — If navigation fails

Use the smallest Explicit Intent first:

```
startActivity(
    new Intent(
        MainActivity.this,
        SecondActivity.class
    )
);
```

Then add extras only after basic navigation works.

STEP 6 — If external Intent fails

Use resolveActivity(). If there is no handler, show a Toast instead of allowing a crash.

STEP 7 — If ListView fails

Use the simplest ArrayAdapter first. Do not start with a custom Adapter unless the question requires custom rows.

```
ArrayList<String> list =
    new ArrayList<>();

list.add("One");
list.add("Two");

ArrayAdapter<String> adapter =
    new ArrayAdapter<>(
        this,
        android.R.layout.simple_list_item_1,
        list
    );

listView.setAdapter(adapter);
```

STEP 8 — If Fragment fails

Use a single FragmentContainerView and a single replace(). Check imports carefully.

STEP 9 — If the examiner asks for more features

Do not rewrite the entire program. Add the feature using the known pattern. For example, Context Menu = registerForContextMenu → onCreateContextMenu → onContextItemSelected.

STEP 10 — Final 30-second checklist

```
[ ] XML file name correct
[ ] setContentView() correct
[ ] Every Java ID exists in XML
[ ] Listener attached
[ ] Validation returns on invalid input
[ ] Adapter attached
[ ] notifyDataSetChanged() after list change
[ ] Activity declared in Manifest
[ ] Intent target class correct
[ ] resolveActivity() for external Intent
[ ] Fragment imports are consistent
[ ] SharedPreferences keys match
[ ] Build again before demonstration
```

BRUTE-FORCE RULE: Compile → Run → Verify one feature → Add next feature → Compile again. Do not add ten features and debug ten problems at once.

22. FINAL ONE-PAGE REVISION

MEMORIZE THESE PATTERNS

```
1. Activity:
setContentView(R.layout.activity_main);

2. View:
Button btn = findViewById(R.id.btn);

3. Read:
String value = et.getText().toString().trim();

4. Validation:
if (value.isEmpty()) {
    et.setError("Required");
    return;
}

5. Click:
btn.setOnClickListener(v -> { });

6. Toast:
Toast.makeText(this, "Saved",
    Toast.LENGTH_SHORT).show();

7. Radio:
int id = rg.getCheckedRadioButtonId();

8. CheckBox:
if (cb.isChecked()) { }

9. Spinner:
spinner.setAdapter(adapter);

10. List:
listView.setAdapter(adapter);

11. Refresh:
adapter.notifyDataSetChanged();

12. Save:
sp.edit().putString("key", value).apply();

13. Read:
sp.getString("key", "default");

14. Alert:
new AlertDialog.Builder(this)
    .setTitle("Confirm")
    .setPositiveButton("Yes", null)
    .setNegativeButton("No", null)
    .show();

15. Explicit Intent:
Intent i = new Intent(this, SecondActivity.class);
i.putExtra("key", value);
startActivity(i);

16. Receive:
getIntent().getStringExtra("key");

17. Browser:
new Intent(Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com"));

18. Dial:
new Intent(Intent.ACTION_DIAL,
    Uri.parse("tel:9876543210"));

19. SMS:
new Intent(Intent.ACTION_SENDTO,
    Uri.parse("sms:9876543210"));

20. Share:
Intent.ACTION_SEND;

21. Dynamic UI:
container.addView(view);

22. Remove:
container.removeView(view);

23. Fragment:
getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.container, new HomeFragment())
    .commit();

24. Back stack:
.addToBackStack(null)
```

```
25. Fragment data:
fragment.setArguments(bundle);
getArguments();

26. Context Menu:
registerForContextMenu(view);

27. Popup Menu:
new PopupMenu(this, anchor);

28. Options Menu:
onCreateOptionsMenu()
onOptionsItemSelected()

29. TextWatcher:
beforeTextChanged()
onTextChanged()
afterTextChanged()

30. Lifecycle:
onCreate
onStart
onResume
onPause
onStop
onRestart
onDestroy
```

THE LIFECYCLE MEMORY TRICK

```
CREATE → START → RESUME
                    ↓
                 PAUSE
                    ↓
                  STOP
                    ↓
               RESTART
                    ↓
            START → RESUME

FINISH / DESTROY:
PAUSE → STOP → DESTROY
```

THE MASTER EXAM MEMORY TRICK

```
INPUT
 ↓
VALIDATE
 ↓
STORE / CONFIRM
 ↓
SEND
 ↓
DISPLAY
 ↓
CLICK
 ↓
LONG CLICK
 ↓
ACTION
```

Final advice: learn the lifecycle and the reusable syntax patterns first. Once those are automatic, most of the model questions become changes to names, arrays, validation rules, messages, destinations and UI widgets rather than completely new programs.